

Reto Técnico – Ingeniero Cloud

Arquitectura de una Aplicación Nativa de Nube

Autor: Christian Cullquicondo

Repositorio GitHub: <https://github.com/ccullquicondo98/cloud-native-azure-demo>

Fecha: 24/12/25

Preguntas teóricas

¿Cuál es la diferencia entre nube pública, privada e híbrida?

Nube pública: Es un modelo donde los recursos de cómputo, almacenamiento y red son provistos por un proveedor externo y compartidos entre múltiples clientes. Ejemplos: Microsoft Azure, AWS, Google Cloud.

Nube privada: La infraestructura es dedicada exclusivamente a una organización, ya sea on-premise o alojada por un proveedor. Ofrece mayor control, pero mayor costo y complejidad operativa.

Nube híbrida: Combina nube pública y privada, permitiendo que cargas de trabajo se ejecuten en ambos entornos. Es común en organizaciones que migran gradualmente a la nube o tienen requisitos regulatorios.

Describe tres prácticas de seguridad en la nube.

1. Principio de mínimo privilegio: Asignar únicamente los permisos necesarios a usuarios, aplicaciones y servicios, reduciendo el impacto de accesos no autorizados.
2. Aislamiento de red: Uso de redes virtuales, subredes y controles de tráfico (NSG, firewalls) para limitar la exposición de servicios críticos.
3. Gestión segura de secretos: Evitar credenciales embebidas en código y utilizar mecanismos seguros como variables de entorno, identidades administradas o servicios de gestión de secretos.

¿Qué es la IaC, y cuáles son sus principales beneficios?, mencione 2 herramientas de IaC y sus principales características.

Infraestructura como Código (IaC) es una práctica que permite definir y administrar infraestructura mediante archivos de configuración versionables, en lugar de configuraciones manuales. Entre sus beneficios principales están:

- Repetibilidad y consistencia
- Automatización de despliegues
- Versionamiento y trazabilidad

Herramientas de IaC seleccionadas:

- Terraform:
 - Multicloud
 - Lenguaje declarativo
 - Gestión del estado de infraestructura

- Amplio ecosistema de proveedores
- Azure Bicep
 - Nativo de Azure
 - Sintaxis simplificada sobre ARM
 - Integración directa con Azure Resource Manager

¿Qué métricas considera esenciales para el monitoreo de soluciones en la nube?

Algunas métricas clave que permiten garantizar rendimiento, estabilidad y eficiencia económica son:

- Disponibilidad: estado de servicios y endpoints
- Latencia: tiempo de respuesta de aplicaciones
- Uso de CPU y memoria: para dimensionamiento adecuado
- Errores y fallos: códigos HTTP, fallos de conexión
- Consumo de recursos: para control de costos

¿Qué es Docker y cuáles son sus componentes principales?

Docker es una plataforma de contenedores que permite empaquetar aplicaciones junto con sus dependencias, garantizando consistencia entre entornos.

Componentes principales:

- Imagen Docker: plantilla inmutable de la aplicación
- Contenedor: instancia en ejecución de una imagen
- Dockerfile: archivo que define cómo construir una imagen
- Registry: repositorio de imágenes

Caso práctico

Selección del proveedor de nube

Para este reto se seleccionó Microsoft Azure como proveedor de servicios en la nube. Por los siguientes motivos:

- Amplio portafolio de servicios PaaS y Kubernetes administrado
- Integración nativa con GitHub y GitHub Actions
- Soporte robusto para contenedores y CI/CD
- Facilidad para implementar redes virtuales y controles de seguridad

Descripción general de la solución

Se diseñó e implementó una aplicación nativa de nube compuesta por:

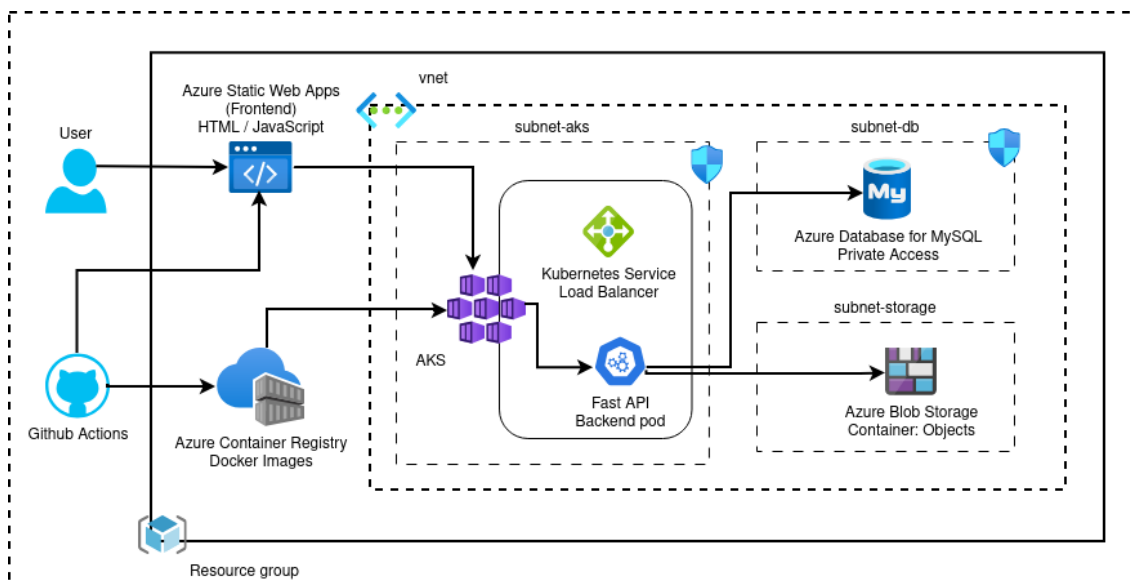
- **Frontend:** Aplicación web estática
- **Backend:** API desarrollada en FastAPI y ejecutada en contenedores
- **Base de datos:** MySQL como servicio administrado
- **Almacenamiento de objetos:** Blob Storage para archivos
- **CI/CD:** GitHub Actions
- **Infraestructura:** Desplegada mediante Terraform

Componentes de la arquitectura

- **Frontend**
 - Azure Static Web Apps
 - Despliegue automático mediante GitHub Actions
 - Consume la API del backend
- **Backend**
 - FastAPI
 - Contenedorizado con Docker
 - Ejecutado en Azure Kubernetes Service (AKS)
- **Base de datos**
 - Azure Database for MySQL Flexible Server
 - Acceso privado desde la red virtual
- **Almacenamiento de objetos**
 - Azure Blob Storage
 - Usado para carga y descarga de archivos desde el backend
- **Registro de contenedores**
 - Azure Container Registry (ACR)
 - Almacena imágenes Docker del backend
- **Red**
 - Virtual Network con subredes separadas para:
 - AKS
 - Base de datos
 - Storage
 - Network Security Groups (NSG) aplicados a nivel de subredes

Diagrama de arquitectura

El diagrama de arquitectura fue elaborado utilizando <https://app.diagrams.net> e incluye:



Justificación del diseño

- **Escalabilidad:** AKS permite escalar horizontalmente el backend.
- **Seguridad:** Acceso privado a base de datos y storage, separación de red.

- **Mantenibilidad:** Uso de IaC y CI/CD.
- **Costo:** Uso de servicios administrados y separación clara de responsabilidades.
- **Buenas prácticas cloud-native:** contenedores, automatización y desacoplamiento.

Repositorio GitHub

El repositorio público contiene:

- Código del backend (FastAPI)
- Configuración Docker
- Infraestructura como Código (Terraform)
- Frontend
- Diagrama de arquitectura
- Este documento en formato PDF